CSC144V Midterm Cheatsheet

Cross-Entropy Loss for Multi-Class Classification

- The SoftMax operator $\sigma: \mathbb{R}^k \to \mathbb{R}^k$ computes a vector of predicted probabilities $\sigma(z): \mathbb{R}^k$ that sum to 1.0, from a vector of logits $z: \mathbb{R}^k$ in the last hidden layer (the penultimate layer), where k is the number of classes:
  - $\sigma(z)_i = \frac{\exp(z_i)}{\sum_{j=1}^{k} \exp(z_j)}$
- The loss function is defined as the negative log likelihood of the predicted probability corresponding to the correct label y:
  - $\text{Loss}(x, y; \theta) = -\log \sigma\big(h_\theta(x)\big)_y = -\log\left(\frac{\exp(h_\theta(x)_y)}{\sum_{j=1}^{k} \exp(h_\theta(x)_j)}\right) = \log\big(\sum_{j=1}^{k} \exp\big(h_\theta(x)_j\big)\big) - h_\theta(x)_y$
  - Minimizing $\text{Loss}(h_\theta(x), y)$ amounts to maximizing the logit $\big(h_\theta(x)\big)_y$ corresponding to the correct label y

Binary Classification Metrics

Consider a medical test that aims to diagnose people with a certain disease. "Positive" denotes sick (has disease), and "Negative" denotes healthy (no disease)
True Positive (TP): a sick person is diagnosed as sick
True Negative (TN): a healthy person is diagnosed as healthy
False Positive (FP): a healthy person is misdiagnosed as sick
False Negative (FN): a sick person is misdiagnosed as healthy

- Precision = $\frac{TP}{TP+FP}$
- Recall (True Positive Rate, TPR) = $\frac{TP}{TP+FN}$
- False Positive Rate (FPR) = $\frac{FP}{FP+TN}$
- Accuracy = $\frac{TP+TN}{TP+TN+FP+FN}$
- $F1 = 2 * \frac{\text{Precision} * \text{Recall}}{(\text{Precision}+\text{Recall})}$

Convolutional Neural Networks

Important

Computation of CONV Layer Sizes

Summary. To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
  - Number of filters K,
  - their spatial extent F,
  - the stride S,
  - the amount of zero padding P.
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
  - $W_2 = (W_1 - F + 2P)/S + 1$
  - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. width and height are computed equally by symmetry)
  - $D_2 = K$
- With parameter sharing, it introduces $F \cdot F \cdot D_1$ weights per filter, for a total of $(F \cdot F \cdot D_1) \cdot K$ weights and K biases.
- In the output volume, the d-th depth slice (of size $W_2 \times H_2$) is the result of performing a valid convolution of the d-th filter over the input volume with a stride of S, and then offset by d-th bias.

Common settings:
K = (powers of 2, e.g. 32, 64, 128, 512)
- F = 3, S = 1, P = 1
- F = 5, S = 1, P = 2
- F = 1, S = 1, P = 0

- A filter typically has square shape $F \times F$. A filter has the same depth D_1 as its input volume, and the number of filters K equals the depth D_2 of its output volume
- Same Padding: stride $S = 1$, filter size $F \times F$, and zero-padding $P = \frac{1}{2}(F - 1)$. Then output feature map has same spatial size as input
  - $W_2 = \frac{1}{S}(W_1 + 2P - F) + 1 = \frac{1}{1}(W_1 + F - 1 - F) + 1 = W_1$; similarly, $H_2 = H_1$
  - e.g., $F = 1 \Rightarrow P = 0$; $F = 3 \Rightarrow P = 1$; $F = 5 \Rightarrow P = 2$

28

Important

Pooling (Sub-Sampling) Layer

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires two hyperparameters:
  - their spatial extent F,
  - the stride S,
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
  - $W_2 = (W_1 - F)/S + 1$
  - $H_2 = (H_1 - F)/S + 1$
  - $D_2 = D_1$
- Introduces zero parameters since it computes a fixed function of the input
- For Pooling layers, it is not common to pad the input using zero-padding.

- A pooling filter has depth 1, and operates over each feature map independently, hence the input volume and output volume have the same depth $D_1 = D_2$
  - In contrast, a CONV filter has the same depth D_1 as its input volume, and the number of filters K equals the depth D_2 of its output volume
  - Common settings: $F = 2, S = 2$, or $F = 3, S = 2$
- Example: pooling w. a 2×2 filter w. stride $S = 2$, no padding. Output volume: $\frac{W_1}{2} \times \frac{H_1}{2} \times D_1$ (since $\frac{1}{2}(W_1 - 2) + 1 = \frac{W_1}{2}, \frac{1}{2}(H_1 - 2) + 1 = \frac{H_1}{2}$)

41

Adversarial Input Generation

- To solve $\min_\theta \mathbb{E}_{(x,y)\sim D}\text{Loss}(x, y; \theta)$ for model training: gradient descent $\theta \leftarrow \theta - \alpha\nabla_\theta\text{Loss}(x, y; \theta)$
  - Update model params θ by following the gradient downhill, in order to decrease $\text{Loss}(x, y; \theta)$. ($\alpha$ is the Learning Rate)

- To solve $\max_{\delta \in \Delta} \text{Loss}(x+\delta, y; \theta)$ for adversarial input generation: gradient ascent $\delta \leftarrow \delta + \alpha \nabla_\delta \text{Loss}(x+\delta, y; \theta)$
  - Update input $x+\delta$ by following the gradient uphill, in order to increase $\text{Loss}(x+\delta, y; \theta)$, while ensuring $\delta \in \Delta$

Adversarial Robust ML

Solve $\min_\theta \mathbb{E}_{(x,y)\sim D} \max_{\delta \in \Delta} \text{Loss}(x+\delta, y; \theta)$

- Inner maximization problem: generating an adversarial input by adding a small perturbation $\boldsymbol{\delta}$ (or ensuring one does not exist).
- Outer minimization problem: training a robust classifier in the presence of adversarial examples. Higher network capacity enables more complex decision boundary and more robust classification

Object Detection: IoU and Non-Max Suppression (NMS)

- Intersection over Union (IoU) measures how much two boxes A and B overlap: **IoU(A, B) = |A ∩ B| / |A ∪ B|** (area of overlap divided by area of union), with $0 \leq \text{IoU} \leq 1$ (0 = no overlap, 1 = identical boxes).
  - For boxes (x_1, y_1, x_2, y_2): intersection width = $\max(0, \min(x_2^A, x_2^B) - \max(x_1^A, x_1^B))$, intersection height likewise; $|A \cap B|$ = width × height; $|A \cup B| = |A| + |B| - |A \cap B|$.
  - A detection counts as a true positive if its IoU with a ground-truth box is at least a threshold (e.g., 0.5).
- **Non-Max Suppression (NMS)** removes duplicate detections of the same object (greedy algorithm):
  - Discard boxes with a low confidence score, then sort the remaining boxes by score.
  - Keep the highest-scoring box and suppress every remaining box whose IoU with it is greater than the threshold (e.g., 0.5). Repeat with the highest-scoring box that remains, until no box is left.
  - Threshold too high: duplicates survive (false positives). Threshold too low: boxes of different, overlapping objects are suppressed (false negatives).
  - In YOLO, each grid cell predicts boxes; IoU and NMS are applied to the output to obtain the final boxes (one per object).